

项目概述

这段Go语言代码实现了K-means聚类算法的可视化程序，使用ebiten游戏引擎展示聚类过程。主要功能包括：

- 生成二维平面上的随机数据点
- 使用K-means算法将数据点聚类到指定的K个类别中
- 可视化展示聚类过程，包括数据点、聚类中心和迭代过程
- 使用动画效果平滑过渡聚类中心的移动
- 显示当前迭代次数和收敛状态

该项目非常适合理解K-means聚类算法的工作原理，通过可视化可以直观地看到算法如何将数据点分组到不同的类别中，以及聚类中心如何随着迭代而移动。

代码结构

主要组成部分

数据结构

定义了Point结构表示数据点，包含X和Y坐标

辅助函数

包括计算欧氏距离和生成随机点的函数

游戏状态

Game结构包含所有点、聚类结果、中心等状态

K-means核心

实现了K-means算法的核心逻辑：分配点和更新中心

动画控制

Update方法控制聚类迭代和动画进度

可视化

Draw方法将数据点和聚类中心绘制到屏幕上

程序执行流程

1. 初始化：生成随机点，随机选择初始聚类中心
2. 游戏循环开始：
 - Update()：执行K-means迭代步骤，更新聚类中心
 - Draw()：绘制当前状态的所有元素，包括数据点和中心

5. 收敛判断：当聚类中心不再变化时，算法收敛
4. 持续渲染：即使收敛后，仍继续渲染可视化结果

核心算法

K-means算法

K-means是一种无监督学习算法，用于将数据点聚类到K个不同的组中。算法目标是最小化每个数据点到其所属聚类中心的距离平方和：

$$\text{minimize } \sum_{i=1}^n \min_{\mu_j \in C} (||x_i - \mu_j||^2)$$

其中：

- n是数据点数量
- C是聚类中心集合
- x_i 是第i个数据点
- μ_j 是第j个聚类中心

算法步骤

1. 随机初始化K个聚类中心
2. 重复以下步骤直到收敛：
 - 分配步骤：将每个数据点分配到最近的聚类中心
 - 更新步骤：计算每个聚类的新中心（平均值）

代码实现

核心算法在stepKmeans()方法中实现：

```
func (g *Game) stepKmeans() bool {
    // 保存当前中心作为上一轮中心（用于动画过渡）
    copy(g.prevCentroids, g.centroids)

    changed := false

    // 1. 分配每个点到最近的聚类中心
    for i, p := range g.points {
        minDist := math.MaxFloat64
        closest := g.clusters[i]

        for j, c := range g.centroids {
            dist := distance(p, c)
            if dist < minDist {
                minDist = dist
                closest = j
            }
        }

        if closest != g.clusters[i] {
            g.clusters[i] = closest
            changed = true
        }
    }

    // 2. 更新聚类中心为每个聚类的平均值
    newCentroids := make([]Point, g.k)
    counts := make([]int, g.k)
```

```

for i, c := range g.clusters {
    newCentroids[c].X += g.points[i].X
    newCentroids[c].Y += g.points[i].Y
    counts[c]++
}

for j := 0; j < g.k; j++ {
    if counts[j] > 0 {
        newCentroids[j].X /= float64(counts[j])
        newCentroids[j].Y /= float64(counts[j])
    }
}

g.centroids = newCentroids
g.iteration++

return changed
}

```

算法特性

K-means算法简单高效，但有以下特性需要注意：

- 结果依赖于初始中心的选择（可能收敛到局部最优）
- 需要预先指定聚类数量K
- 对噪声和离群点敏感
- 适合发现球形聚类

可视化部分

绘制元素

数据点

根据所属聚类，使用不同颜色绘制：

- 每个点根据最近的聚类中心分配颜色
- 点的大小为4x4像素
- 使用循环颜色数组确保不同聚类有明显区分

聚类中心

使用黑色方块表示，带有动画过渡效果：

- 中心大小为8x8像素
- 使用动画平滑过渡到新位置
- 动画进度由animProgress控制

动画控制

动画效果通过Update()方法控制：

```

func (g *Game) Update() error {
    if g.converged {
        return nil
    }
}

```

```

// 动画进行中，更新进度
if g.animProgress < 1.0 {
    g.animProgress += g.animSpeed
    if g.animProgress > 1.0 {
        g.animProgress = 1.0
    }
    return nil
}

// 动画完成，执行下一步K-means迭代
changed := g.stepKmeans()
if !changed {
    g.converged = true
}
g.animProgress = 0 // 重置动画进度

return nil
}
```

关键动画逻辑：

- 使用animProgress变量跟踪动画进度（0-1）
- 每次更新增加animSpeed（0.01）
- 动画完成后执行K-means迭代步骤
- 在Draw()中根据animProgress插值计算中心位置

坐标映射

将数学坐标（0-100范围）映射到屏幕坐标：

```

// 绘制所有点（按聚类颜色区分）
for i, p := range g.points {
    clusterID := g.clusters[i]
    c := colors[clusterID%len(colors)]

    // 坐标映射到窗口尺寸
    x := int(p.X * float64(g.width) / 100)
    y := int(p.Y * float64(g.height) / 100)

    // 绘制点（4x4的方块）
    for dx := -2; dx <= 2; dx++ {
        for dy := -2; dy <= 2; dy++ {
            screen.Set(x+dx, y+dy, c)
        }
    }
}
}
```

这种映射确保生成的随机点（0-100范围）能够正确显示在窗口内。

参数说明

关键参数

参数	含义	默认值	调整建议
k	聚类数量	5	根据数据分布调整，过小会导致欠拟合，过大会导致过拟合
width, height	窗口尺寸	800x600	根据需要调整窗口大小
animSpeed	动画速度	0.01	值越大动画越快，可调整为0.005-0.05之间
count	数据点数量	300	数据越多，聚类效果越明显，但计算量也越大

min, max 数据点生成范围 0, 100 通常不需要调整，除非需要特殊分布的数据

初始化参数

在main()函数中可以调整的参数：

```
func main() {  
    // 生成300个随机点（范围0-100）  
    points := generateRandomPoints(300, 0, 100)  
  
    // 聚类数量  
    k := 5  
  
    // 初始化游戏（包含动画逻辑）  
    game := NewGame(points, k)  
    ebiten.SetWindowSize(game.width, game.height)  
    ebiten.SetWindowTitle("K-means 聚类过程动画")  
  
    // 运行动画  
    if err := ebiten.RunGame(game); err != nil {  
        panic(err)  
    }  
}
```

通过修改这些参数，可以观察不同条件下K-means算法的表现。